



Symbolic Simulation



Traditional Simulation

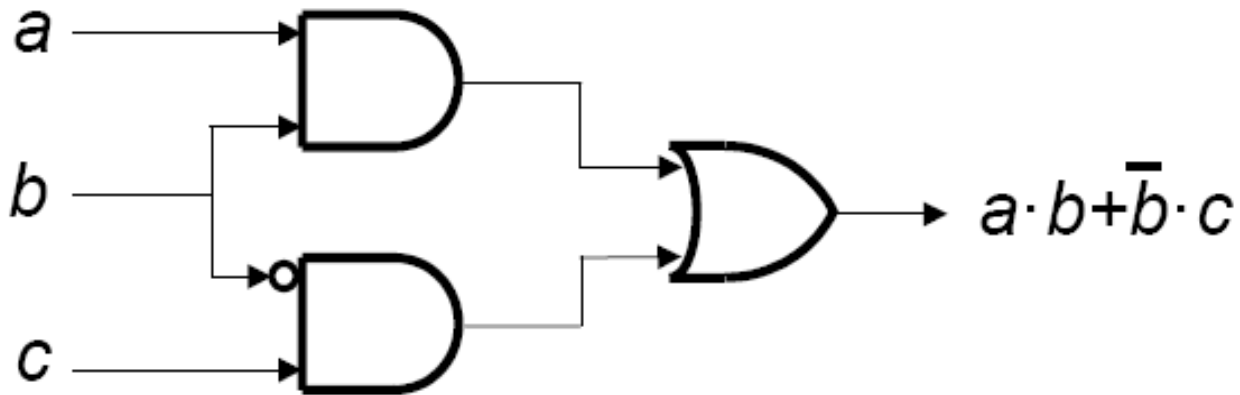
- $(x+1)^2 = x^2 + 2x + 1$?

x	$(x + 1)^2$	$x^2 + 2x + 1$
0	1	1
1	4	4
2	9	9
3	16	16
9	100	100
67	4624	4624
...	...	...

- Use the concrete values to enumerate all the inputs.

Symbolic Simulation

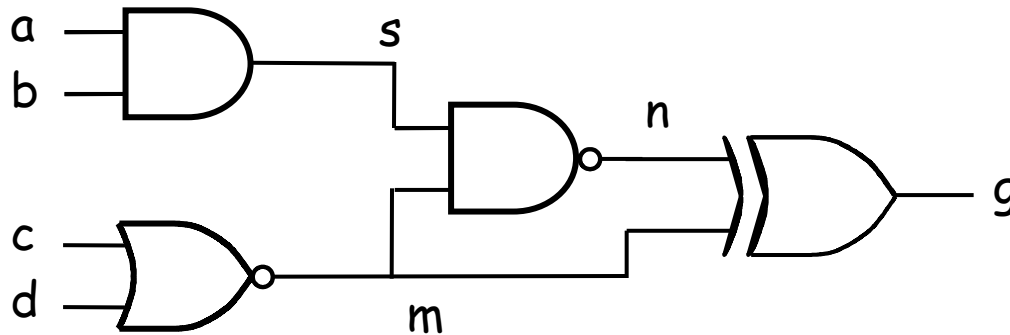
- Use the symbols to encode the history of computing.



Symbolic Simulation

- Symbolic simulation
 - Use symbolic, rather than concrete, data
 - Extract logical conditions for path feasibility
 - Reason using decision procedures
- Recent successes with symbolic simulation
 - Symbolic trajectory evaluation
 - Microcode verification
 - Static analysis for dynamic errors
 - Verification of Java programs

Symbolic Simulation for Combinational Circuits

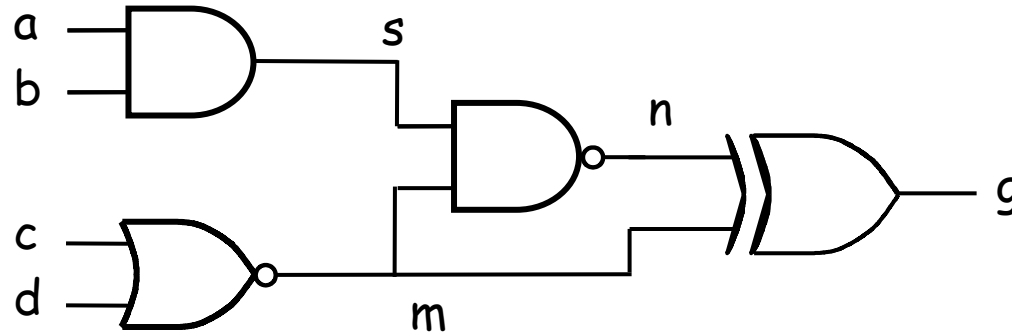


And Gate	Nor Gate	Nand Gate	Xor Gate
1ns	2ns	2ns	3ns

	t+1	t+2	t+3	t+4	t+5	t+6	t+7
s							
m							
n							
g							

- Delays of gates are given as follows:
- At time t , the initial inputs are (a_0, b_0, c_0, d_0) .
- At time $t+1$, the inputs are (a_1, b_1, c_1, d_1) .
- At time $t+k$, the inputs are (a_k, b_k, c_k, d_k) .
- Building a table for $h(i) \oplus h(i+1)$ for each circuit node h as we discussed in the class.
- The table can be used for detecting switching activities.

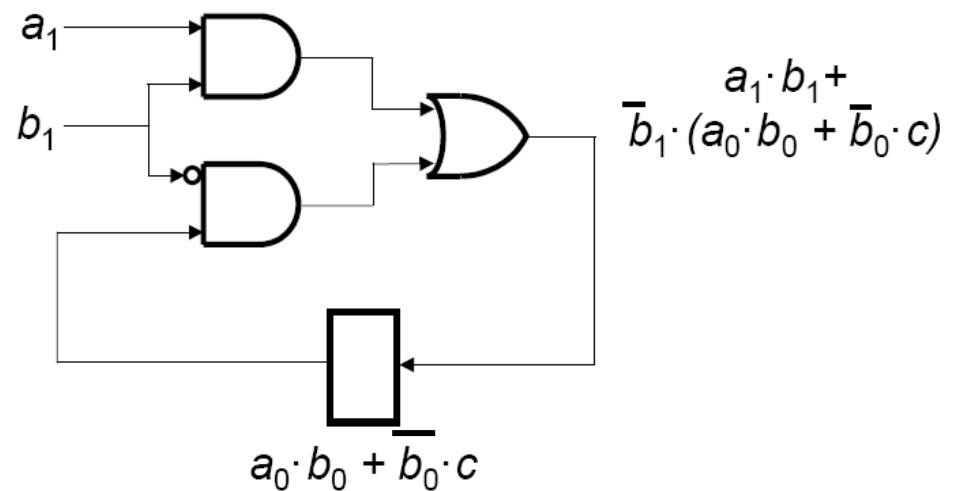
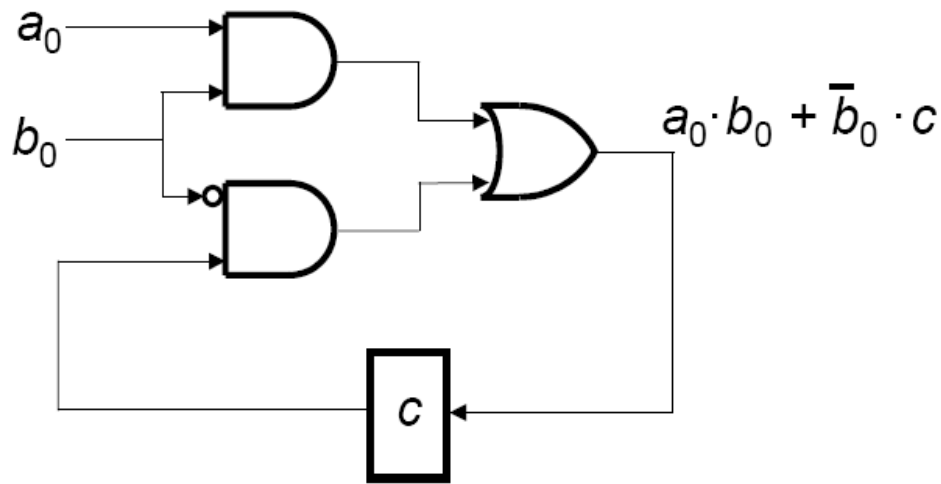
Combinational Circuits with Delays



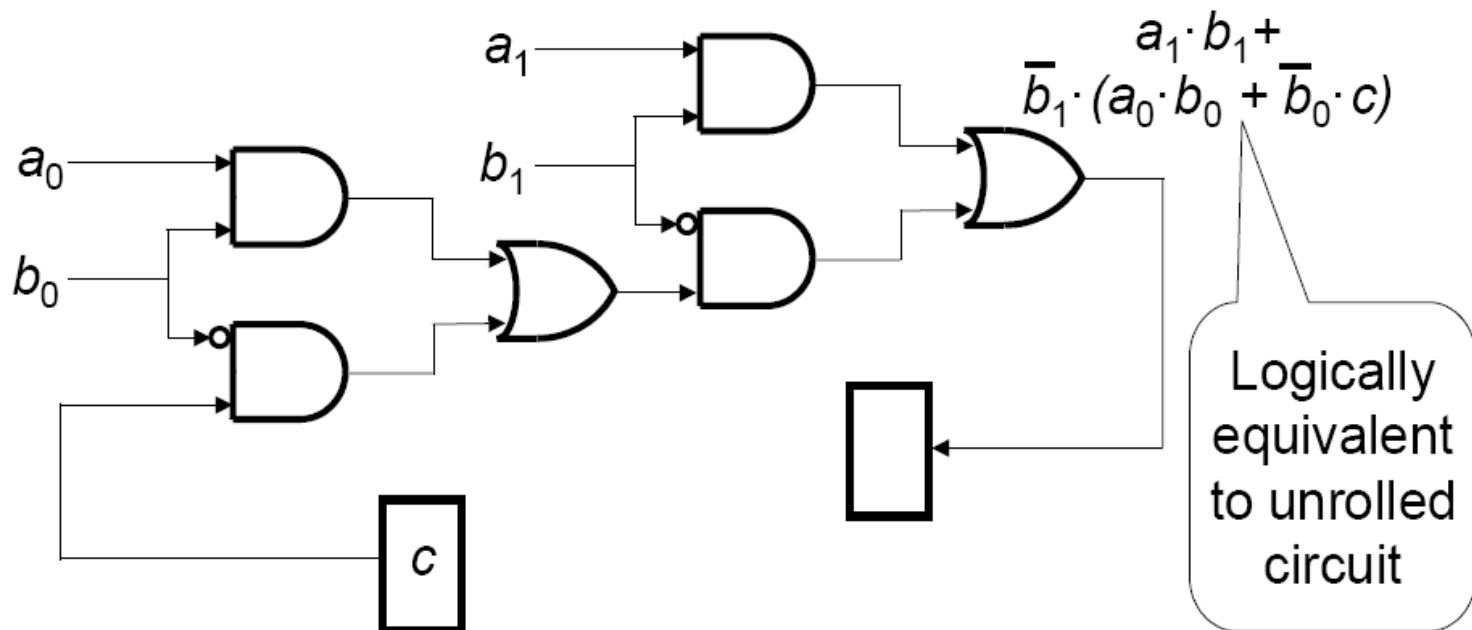
And Gate	Nor Gate	Nand Gate	Xor Gate
1ns	2ns	2ns	3ns

	t	t+1	t+2	t+3	t+4	t+5	t+6	t+7	t+8
s	X	a_0b_0	a_1b_1	a_2b_2	a_3b_3	a_4b_4	a_5b_5	a_6b_6	a_7b_7
m	X	X	$\overline{c_0 + d_0}$	$\overline{c_1 + d_1}$	$\overline{c_2 + d_2}$	$\overline{c_3 + d_3}$	$\overline{c_4 + d_4}$	$\overline{c_5 + d_5}$	$\overline{c_6 + d_6}$
n	X	X	X	X	$\overline{(a_1b_1)(\overline{c_0 + d_0})}$	$\overline{(a_2b_2)(\overline{c_1 + d_1})}$	$\overline{(a_3b_3)(\overline{c_2 + d_2})}$	$\overline{(a_4b_4)(\overline{c_3 + d_3})}$	$\overline{(a_5b_5)(\overline{c_4 + d_4})}$
g	X	X	X	X	X	X	X	$\overline{(a_1b_1)(\overline{c_0 + d_0})} \oplus \overline{c_2 + d_2}$	$\overline{(a_2b_2)(\overline{c_1 + d_1})} \oplus \overline{c_3 + d_3}$

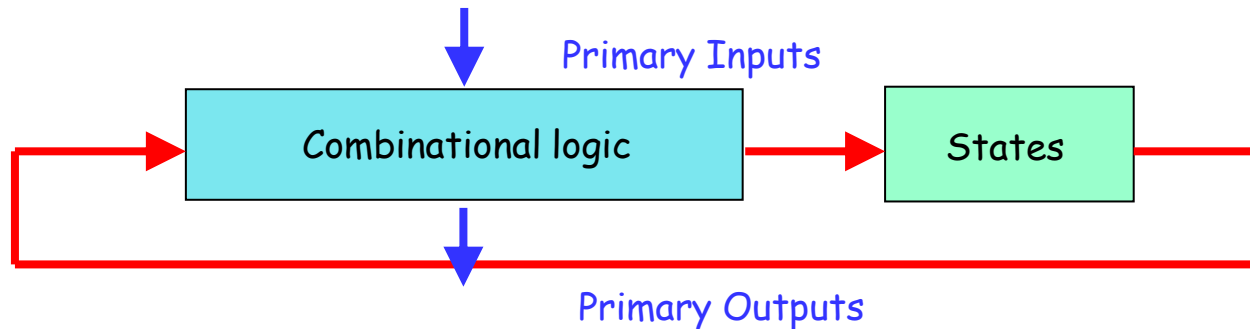
Symbolic Simulation for Sequential Circuits



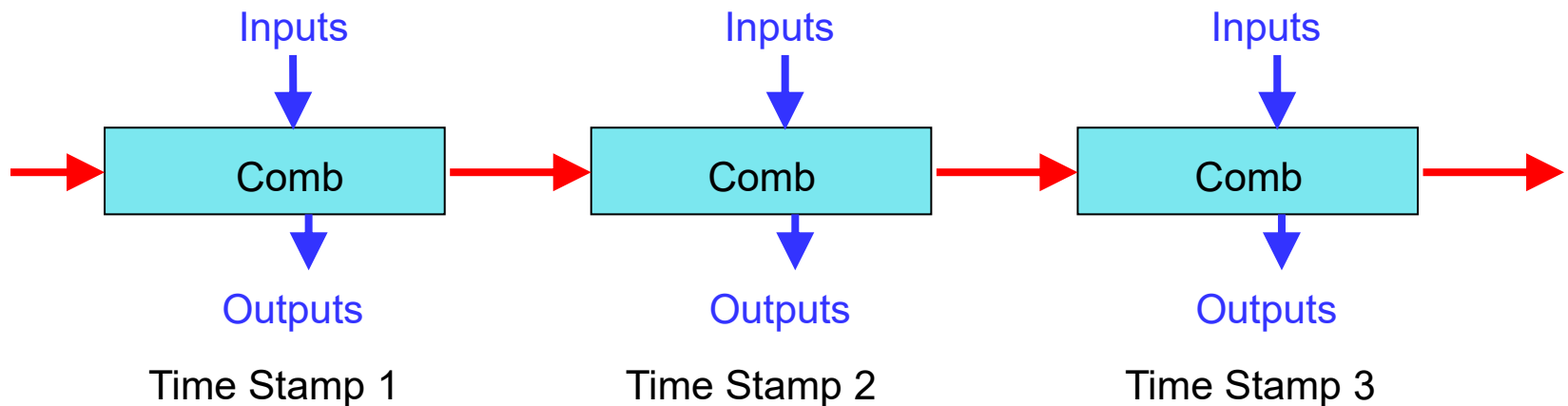
Sequential Circuits as “Unrolling”



Sequential -> Combinational

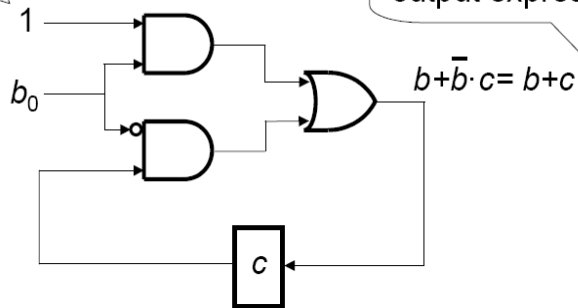


- A sequential circuit can be unfolded or unrolled into a chain of combinational circuits with timing stamps.



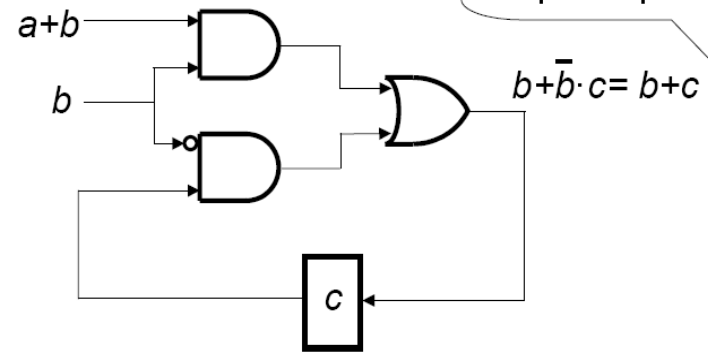
Symbolic Simulation

Some inputs
can be constants



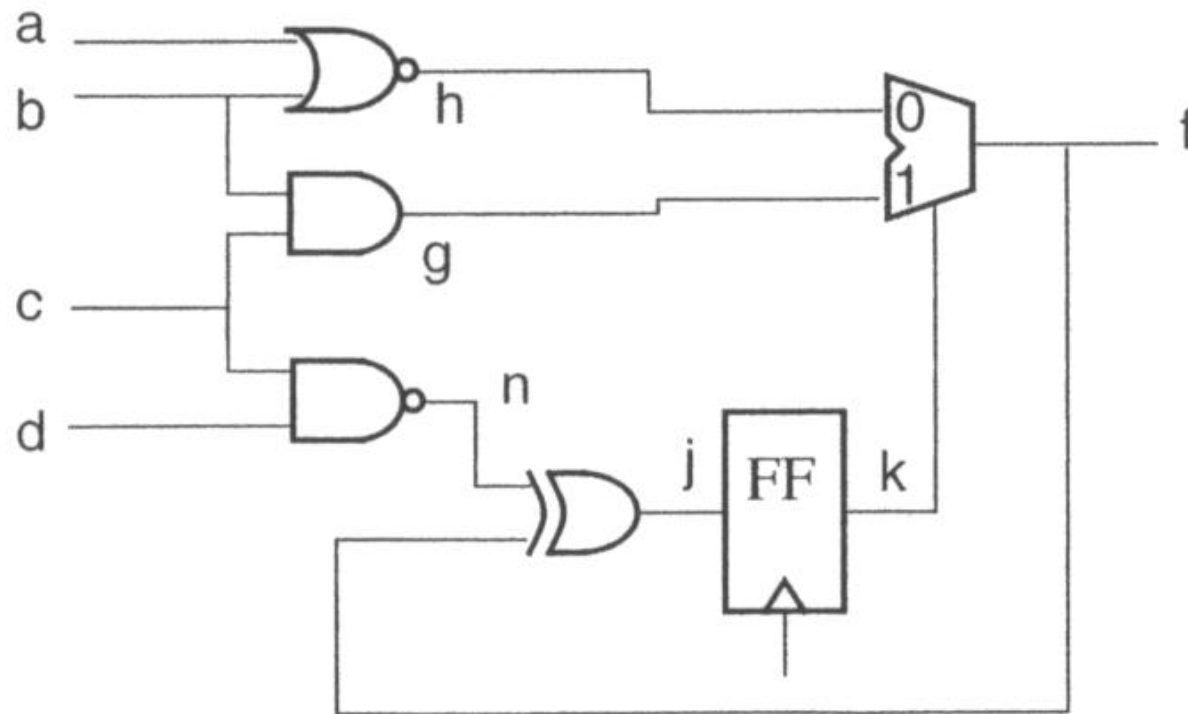
Resulting in simpler
output expressions

Input expressions
can also be related

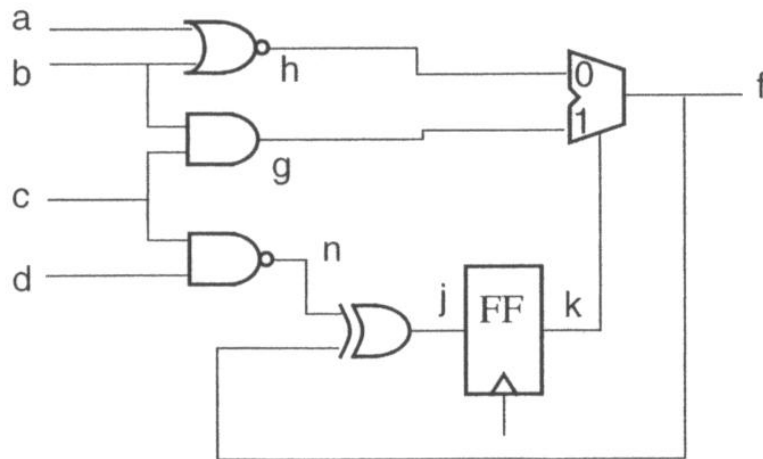


Resulting in simpler
output expressions

Example [3]



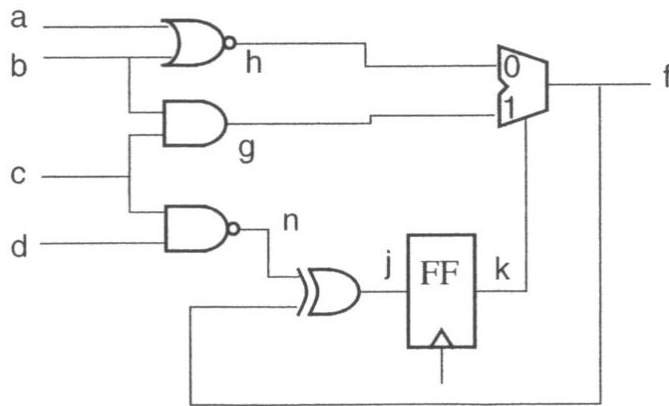
Example [3]



Symbolic Simulation Result at Cycle 1

Node	Cycle 1
h	$\overline{a_1 + b_1}$
g	$b_1 \cdot c_1$
k	k_1
f	$\bar{k}_1(\overline{a_1 + b_1}) + k_1(b_1 \cdot c_1)$
n	$\overline{c_1 \cdot d_1}$
j	$(\overline{c_1 \cdot d_1}) \oplus (\bar{k}_1(\overline{a_1 + b_1}) + k_1(b_1 \cdot c_1))$

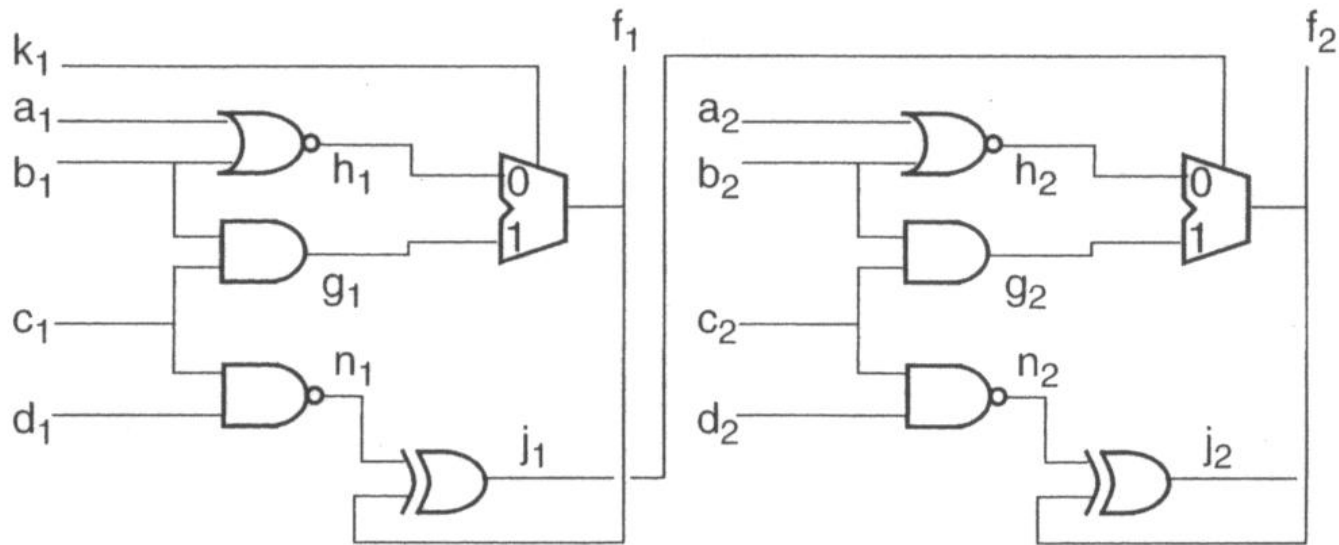
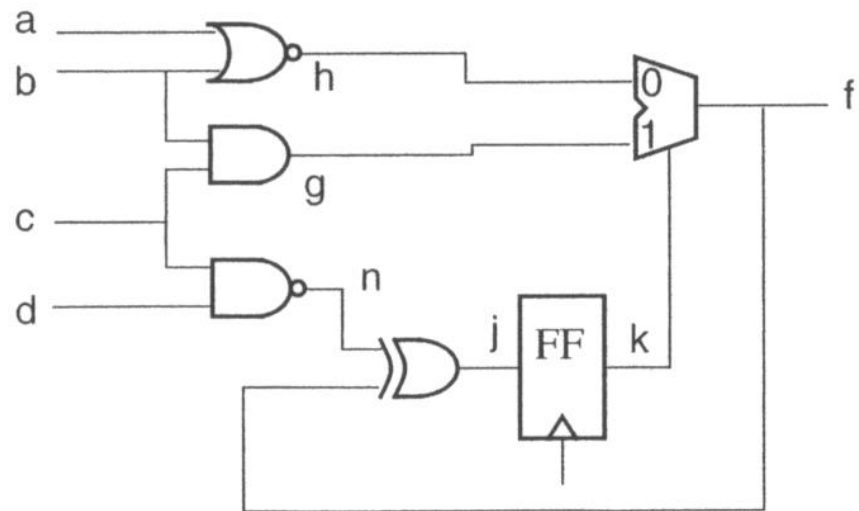
Example [3]



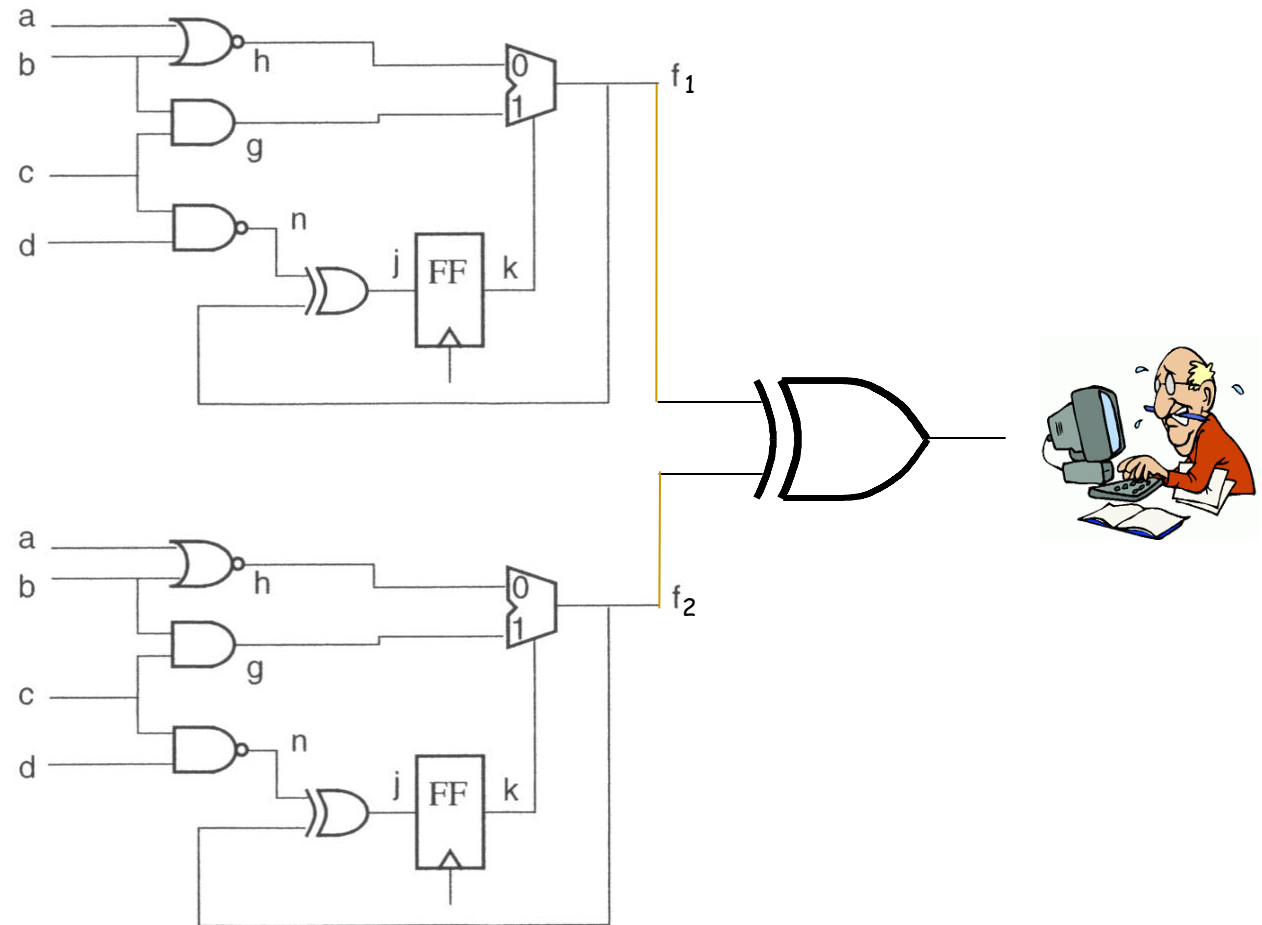
Symbolic Simulation Result at Cycle 2

Node	Cycle 2
h	$\overline{a_2 + b_2}$
g	$b_2 \cdot c_2$
k	$(\overline{c_1 \cdot d_1}) \oplus (\bar{k}_1(\overline{a_1 + b_1}) + k_1(b_1 \cdot c_1))$
f	$\overline{((\overline{c_1 \cdot d_1}) \oplus (\bar{k}_1(\overline{a_1 + b_1}) + k_1(b_1 \cdot c_1)))(\overline{a_2 + b_2}) + ((\overline{c_1 \cdot d_1}) \oplus (\bar{k}_1(\overline{a_1 + b_1}) + k_1(b_1 \cdot c_1)))(b_2 \cdot c_2)}$
n	$\overline{c_2 \cdot d_2}$
j	$\overline{((\overline{c_1 \cdot d_1}) \oplus (\bar{k}_1(\overline{a_1 + b_1}) + k_1(b_1 \cdot c_1)))(\overline{a_2 + b_2}) + ((\overline{c_1 \cdot d_1}) \oplus (\bar{k}_1(\overline{a_1 + b_1}) + k_1(b_1 \cdot c_1)))(b_2 \cdot c_2)} \oplus (\overline{c_2 \cdot d_2})$

Sequential to Combinational

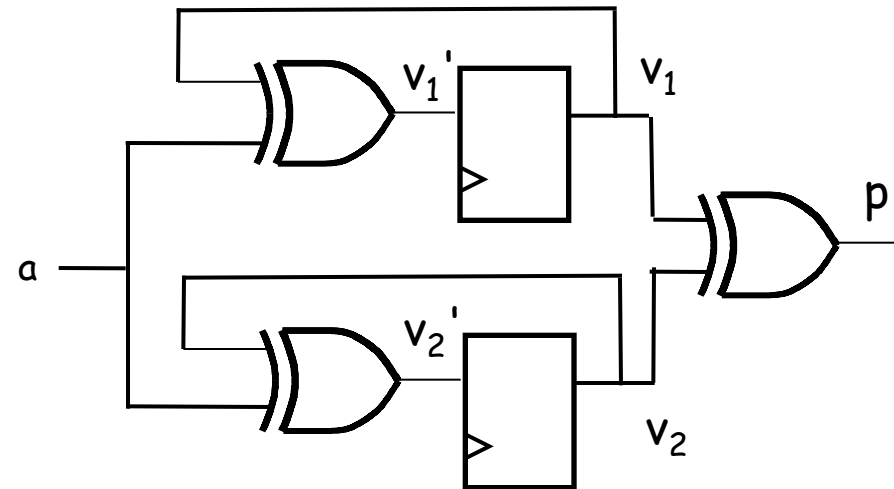


Equivalence Checking by Symbolic Simulation



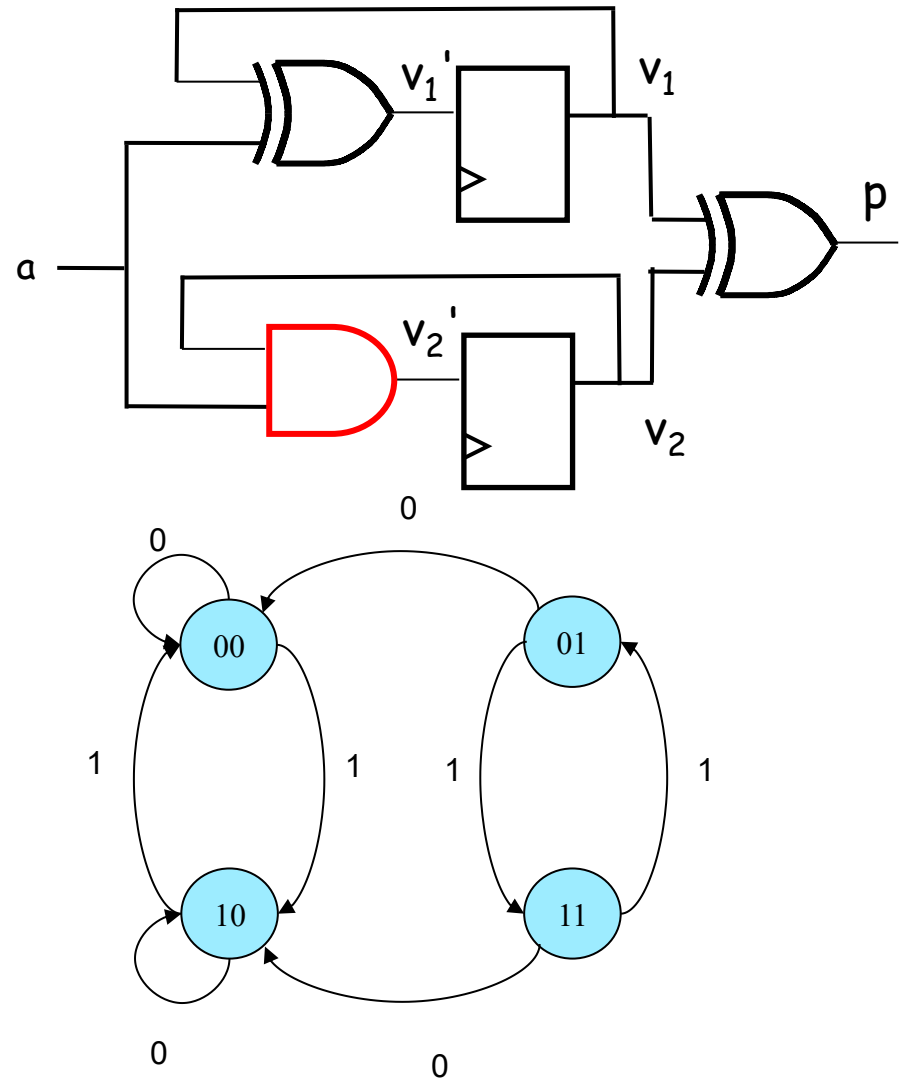
Equivalence Checking by Symbolic Simulation

- Initial state: $\langle v_1 = m, v_2 = m \rangle$
- Cycle 1: input = a_1
 - $p_1 = (m \oplus a_1) \oplus (m \oplus a_1)$
 - $p_1 = 0$
- Cycle 2: input = a_2
 - $p_2 = ((m \oplus a_1) \oplus a_2) \oplus ((m \oplus a_1) \oplus a_2)$
 - $p_2 = 0$
- Depth of a sequential circuit = 1
- (Diameter)

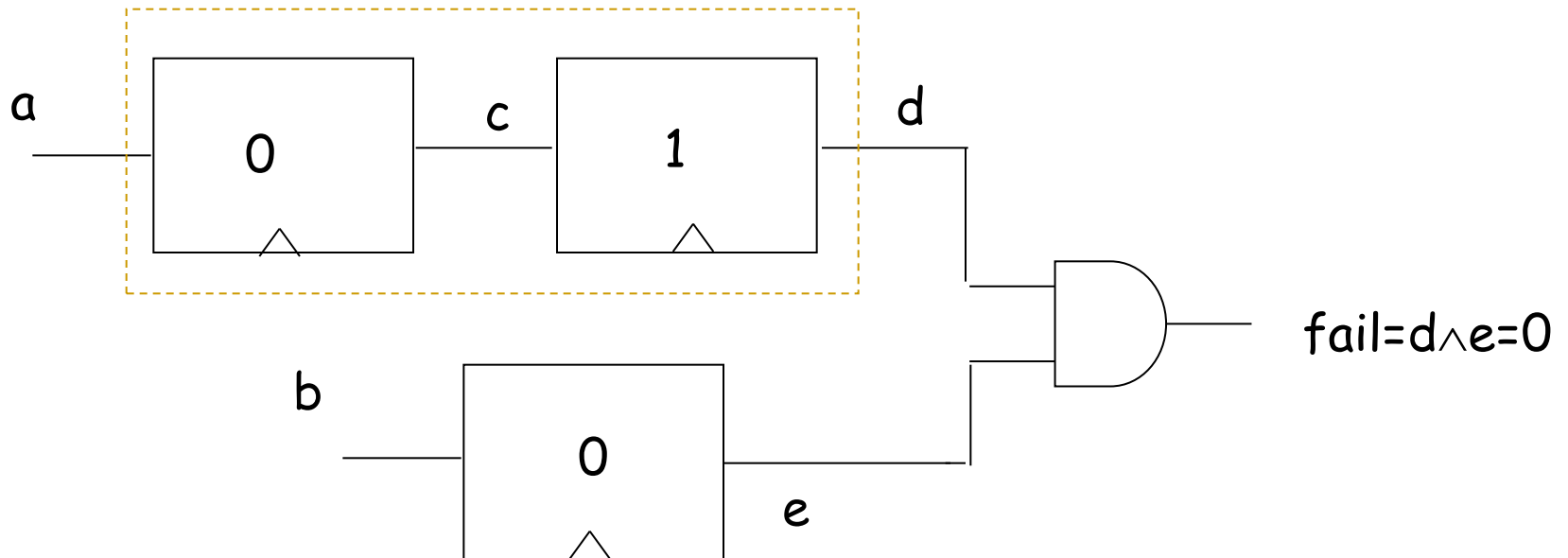


Equivalence Checking by Symbolic Simulation

- Initial state: $\langle v_1=m, v_2=m \rangle$
- Cycle 1: input = a_1
 - $p_1 = (m \oplus a_1) \oplus (ma_1)$
 - $p_1 = m + a_1$
 - It is satisfiable !
 - $m=1$ or $a_1=1$

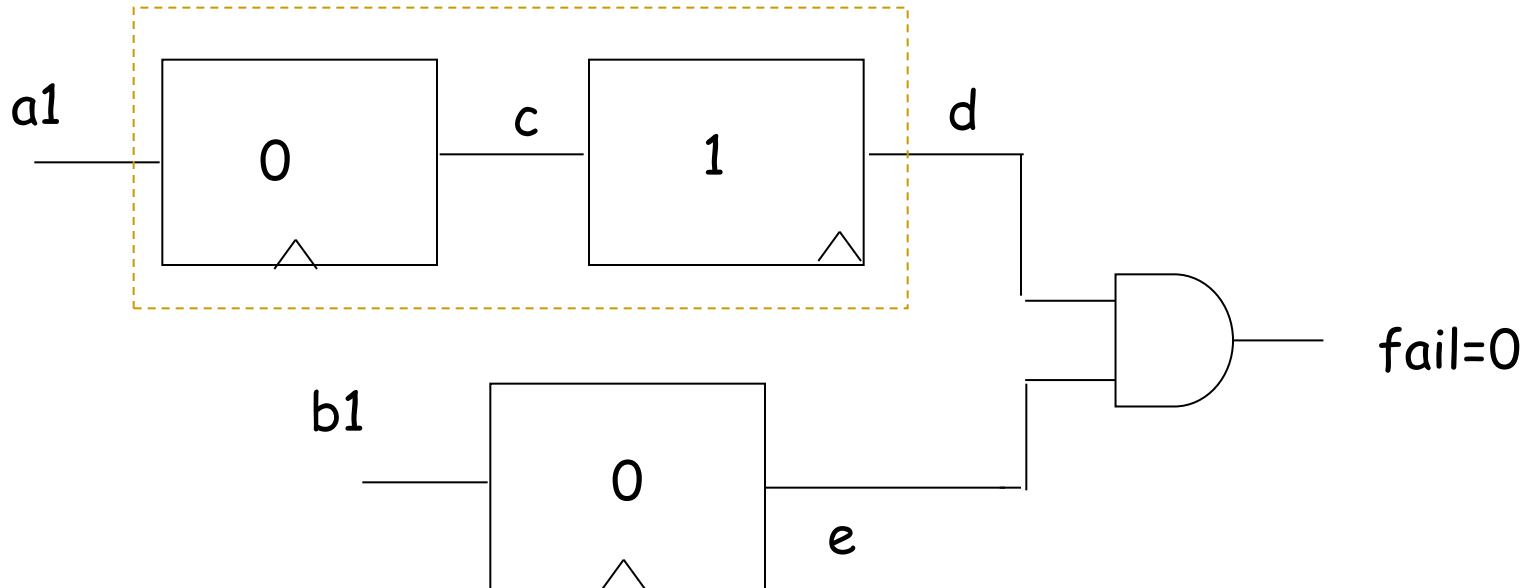


Symbolic Simulation



- Initial State = $\langle c, d, e \rangle = \langle 0, 1, 0 \rangle = !c \ \& \ d \ \& \ !e$
- Output = fail ($\langle 0, 1, 0 \rangle$) = 0

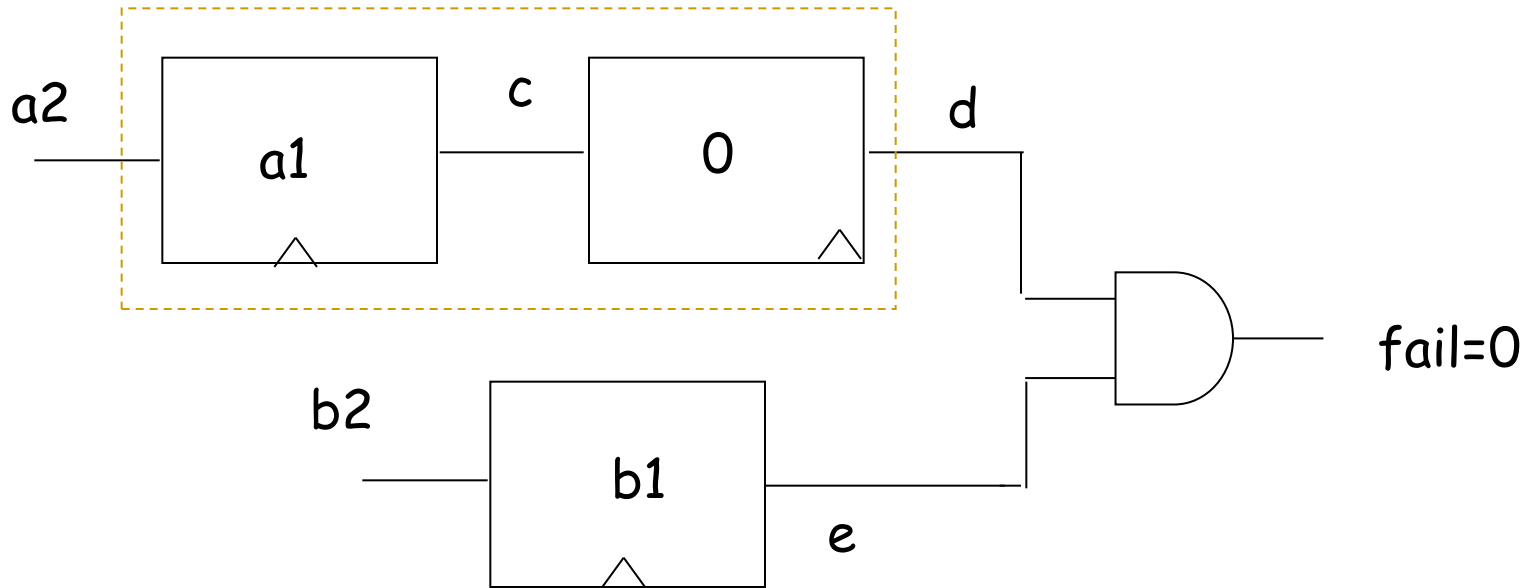
Symbolic Simulation



Cycle 0: (Before the arrival of the rising edge)

Initial State = $\langle 0, 1, 0 \rangle$, Input = $\langle a1, b1 \rangle$, Output = $\text{fail} = 0 \wedge 1 = 0$

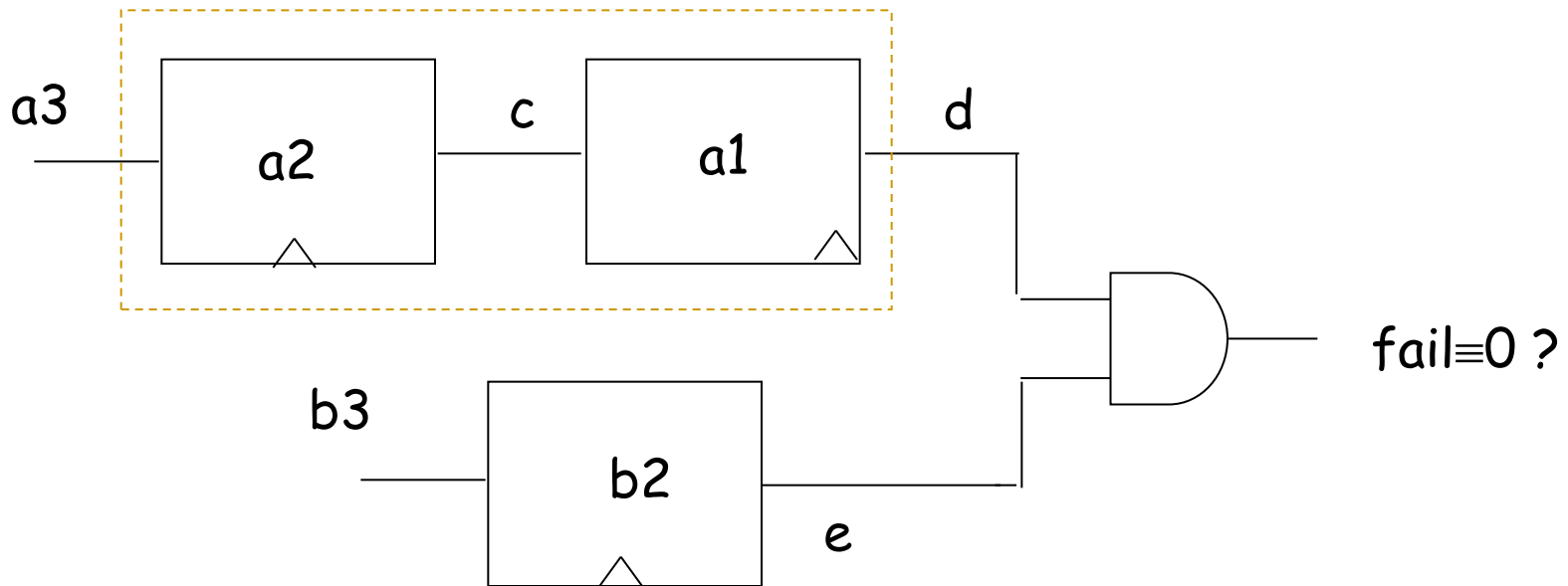
Symbolic Simulation



Cycle 1: (Before the arrival of the rising edge)

State = $\langle a1, 0, b1 \rangle$, Input = $\langle a2, b2 \rangle$, Output = $fail = b1 \wedge 0 = 0$

Symbolic Simulation



- Cycle 2: Hit the fail state where $\text{fail}=1$
- State = $\langle a_2, a_1, b_1 \rangle$, Input = $\langle a_3, b_3 \rangle$, Output = $\text{fail} = a_1 \wedge b_2$
- $(a_1=1), (b_2=1) \rightarrow \text{fail}=1$

References

- E. M. Clark, O. Grumberg and D. A. Peled, "Model Checking", MIT Press, 1999.
- M. Huth, M. Ryan, "Logic in Computer Science: Modeling and Reasoning about Systems", Cambridge University Press, 2000.
- William K. Lam, "Hardware Design Verification: Simulation and Formal Method-Based Approaches", Prentice Hall, 2005.